

Оптимизация конфигурации PostgreSQL на основе алгоритма SMAC

Направление 09.03.04 Программная инженерия

Студент: Ганиуллин Р.Р., гр. 5130904/20104

Руководитель: Прокофьев О.В., старший преподаватель

СПбПУ Петра Великого, Высшая школа программной инженерии

26 мая 2026 г.

Проблема и метод

Актуальность

Проблема: производительность PostgreSQL критически зависит от конфигурации

Инструмент	Учёт нагрузки	Адаптация	Открытый и актуальный
Ручная настройка	✓	✓	—
pgTune	✗	✗	✓
OtterTune	✓	✓	✗
Разработанная система	✓	✓	✓

Цель и задачи

Цель: разработать прототип системы автоматической оптимизации конфигурации PostgreSQL без участия эксперта

Задачи:

1. Анализ предметной области и существующих инструментов
2. Выбор и обоснование метода оптимизации
3. Разработка архитектуры и реализация на Python
4. Сравнительная экспериментальная проверка (SMAC vs TPE vs pgTune)

Выбор метода оптимизации

PostgreSQL: более 300 параметров конфигурации. Целевая функция — чёрный ящик:

- аналитический вид неизвестен
- градиентные методы неприменимы
- смешанные типы значений: целочисленные, вещественные, категориальные
- одна оценка — полноценный прогон нагрузки (минуты)

Метод	Учёт истории	Итераций до оптимума
Перебор по сетке	✗	МИЛЛИОНЫ
Случайный поиск	✗	ТЫСЯЧИ
Байесовская оптимизация	✓	20–60

Байесовская оптимизация

Задача: $\min_{\mathbf{x} \in \mathcal{X}} f(\mathbf{x})$, где f — дорогостоящая функция чёрного ящика

Цикл оптимизации:

1. Построить суррогатную модель $\hat{f}$ на истории испытаний $\mathcal{D}$
2. Максимизировать функцию приобретения $\alpha(\mathbf{x}) \rightarrow$ кандидат $\mathbf{x}_{n+1}$
3. Провести испытание: запустить `rgbench`, получить TPS / задержку
4. Обновить $\mathcal{D}$; повторить — оптимум за 20–60 итераций

Алгоритм SMAC

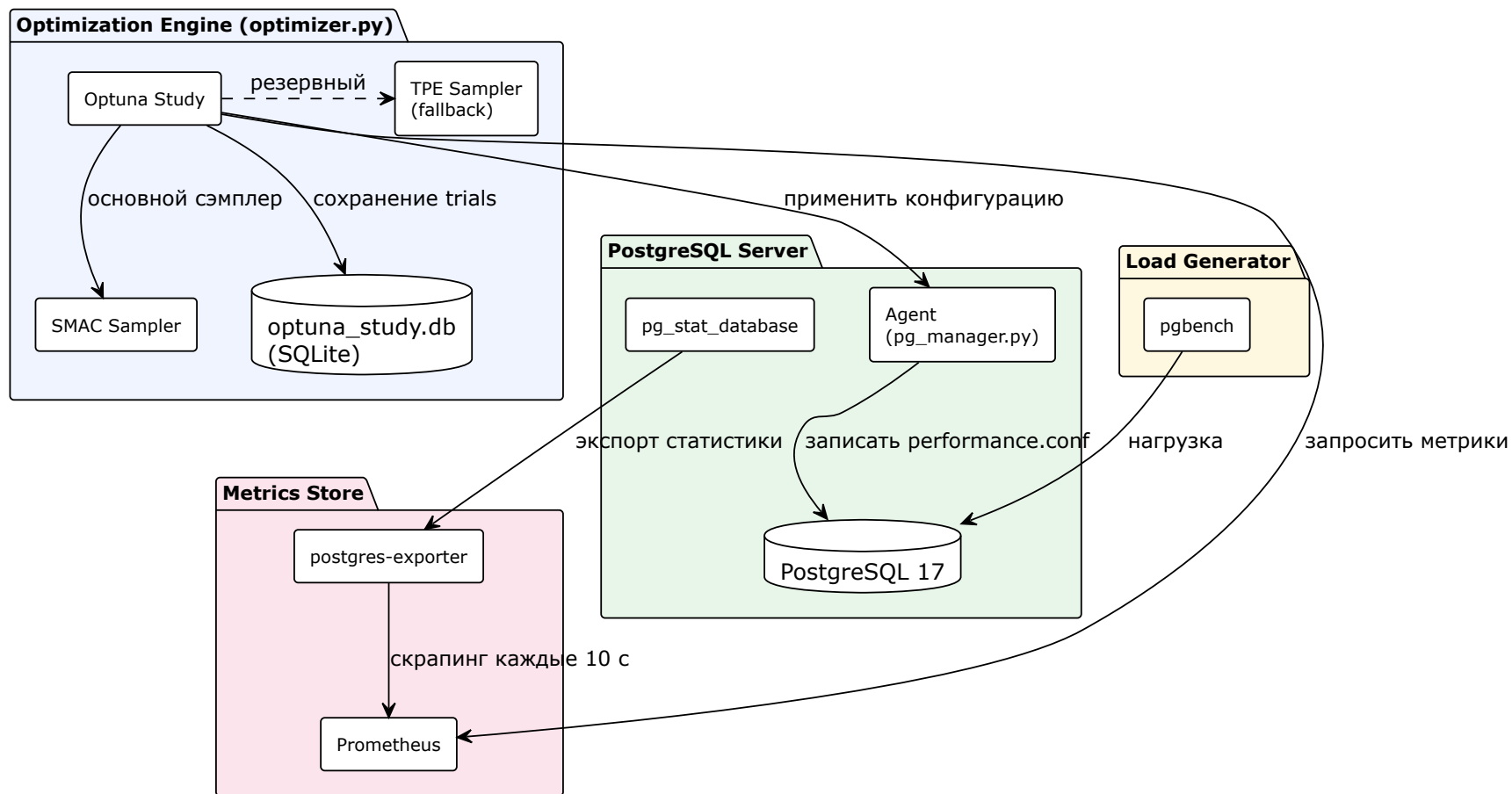
Sequential Model-based Algorithm Configuration

Суррогат	случайный лес — нативно работает с категориальными, целочисленными и вещественными параметрами
Интенсификация	новый кандидат сравнивается с текущим лучшим — снижает шум разовых оценок
Сравнение с TPE	строит явную модель пространства; TPE только разделяет «хорошие» и «плохие» точки

Реализован как сэмплер Optuna ([optunahub](https://optuna.github.io/)) — алгоритм легко заменить без переписывания

Система и эксперименты

Архитектура системы



OLTP и OLAP

Характеристика	OLTP	OLAP
Запросы	Короткие, точечные	Длинные, аналитические
Пример	Банковские транзакции	Квартальные отчёты
Целевая метрика	TPS, задержка	Пропускная способность
Узкое место	work_mem, WAL	shared_buffers, параллелизм

Оптимальные конфигурации PostgreSQL существенно различаются — в работе оба профиля тестировались отдельно

Методология экспериментов

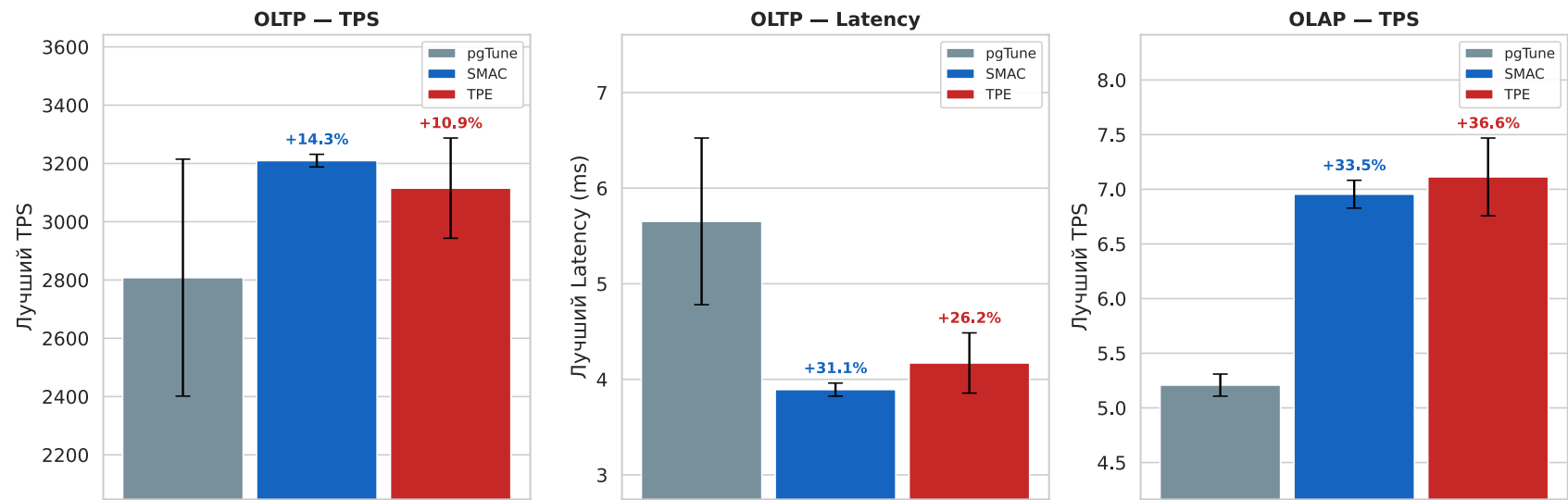
Среда:

- 6-ядерный CPU
- контейнер PostgreSQL (ядра 2–5, 4 ГБ RAM, SSD NVMe)
- pgbench scale = 50 , 20 клиентов, 120 с на испытание

Сценарий	Целевая метрика	Сэмплеры
OLTP	TPS (максимум)	SMAC и TPE, по 60 итераций
OLTP	Задержка (минимум)	SMAC и TPE, по 60 итераций
OLAP	TPS (максимум)	SMAC и TPE, по 60 итераций

База сравнения: конфигурация pgTune, добавленная нулевым испытанием

Результаты



Анализ результатов

Прирост относительно pgTune:

Сценарий	SMAC	TPE
OLTP TPS	+14,3%	+10,9%
OLTP задержка	-31,1%	-26,2%
OLAP TPS	+33,5%	+36,6%

Сходимость: 70% прироста — уже в первых 20 итерациях из 60

Ключевые параметры: shared_buffers → 1536–2048 МБ;
checkpoint_completion_target ≥ 0,9; JIT отключён для OLTP

Заключение

Выводы

1. **Разработана** система автоматической оптимизации конфигурации PostgreSQL
2. **Достигнуто** до **+14,3% TPS** и **-31,1% задержки** относительно pgTune
3. **Подтверждена** применимость байесовской оптимизации: оба алгоритма существенно превзошли pgTune
4. **Открытый инструмент:** Docker + Optuna + Prometheus

Направления развития: онлайн-адаптация при смене нагрузки; механизм авто-отката; тестирование на реальных трассах

Спасибо за внимание!

Готов ответить на вопросы